

What is openly readable in a Sony ILCE-7SM3 (a7S III) firmware update file, and where the trust boundary really sits

A static audit of a single file: measurements, limits, and the errors of
the instruments that took them

Oxunjon Ubaydullayev* Aiodam (autonomous research agent)[†]

4 October 2026

Abstract

We take apart a single public Sony firmware update file, `BODYDATA.DAT`, for the camera ILCE-7SM3 (a7S III): **743 818 632** bytes, sha256 `dbdd8b02...c9eb`. The work is a static audit of the file in an isolated environment, complemented by exactly one bounded observation of the live device: TCP probes of three ports and a single SSH handshake, with no authentication, no payload written to the camera, and nothing sent to any other host. The result has two halves, and both were measured rather than deduced.

First: nothing inside the file is protected by anything the file carries. The container is checked by a CRC32 (`0x1fafdcaa`) that any reader can recompute without a key, and so is the integrity of the image contents. The firmware header is encrypted with a shared *public* key in ECB mode, so it is readable by anyone holding the published key; and the mode itself leaks structure: across all 710 MiB of ciphertext there is exactly **1** run of identical 16-byte blocks (**26** blocks, starting at byte **96**), and that run was predicted from the decrypted header *before* it was measured. The body (99.99993 % of the file) is decrypted with a **public** key published in an open but unmerged pull request, yielding a stream of **740 912 128** bytes, sha256 `11880291...1e66`, which contains the updater’s own file-system image (squashfs, **58** files), a **159**-member firmware tar, and **1** unencrypted OpenSSH private key (ECDSA P-256) shipped inside a publicly distributed image.

Second: no signature inside the file is verified by anything the file carries. Public RSA keys are present — **6** distinct moduli — but none of them verifies the 256-byte block at the tail ($m[0:2] = 49\ 7f$: no PKCS#1 v1.5 structure), and no verification key is present at all. The updater’s own pipeline map (`config/config.xml`) consists of steps of the form “check the CRC32 of a group, then run its module”; there is no signature step in it. The only honest conclusion is that the *trust boundary lives outside the file, in the device*, and cannot be probed from a file.

*Commissioned the audit and wrote its terms of reference; performed the measurements on the live device (reading the camera’s host key and scanning his own network) and supplied the raw logs; decides what is published.

[†]The measurements, the instruments, the format analysis and this text are its work. The errors of the instruments are not deleted from the record; they are listed in §11.

One result came from the live device. The camera presents a host key that matches the key inside the file in none of three comparison routes (they differ in **64** of the **66** bytes of the point), while negotiating *exactly* the algorithm set that the firmware’s own `sshd_config` imposes. So the firmware’s `sshd` is indeed running on the camera, and its host key is its own — which means the private key shipped inside the public file is *not* this camera’s live identity, although it remains a private key in an open distribution. What could not be established is whether that identity is stable at all: whether the live key survives a power cycle was *not* measured, and neither was whether it is the same on every camera of the model. If it is created afresh on every boot, a client cannot pin the camera and the cryptography is present while the authentication is not.

What this work does *not* establish is as much a part of the result as what it does: signature verification on the device (the verification key is not in the file), rollback protection, the parser’s robustness against malformed input, behaviour on an interrupted update, and the debug interfaces of a release build all need a camera, a boot-loader dump or service mode — not a file. The trust boundary sits outside the file.

Finally, the paper is checked by an instrument rather than taken on trust. Every number is re-derived from its artifact’s bytes (`paper_check.py`); a full re-verification of all findings by a separate instrument yields **47** green checks out of **47** with **5** controls that must fail, and finds **4** corrections to numbers recorded in earlier reports.

Contents

1	Subject and scope: what was checked, and what it cannot be	3
2	Method: the rules this work followed	3
3	The container: parsed to the byte, but not authenticated	4
4	The header is readable with a public key, and ECB leaks structure	4
5	The body: opened with a public key; the “unaccounted bytes” were arithmetic	5
6	What is inside	6
6.1	The updater’s file-system image	6
6.2	The update pipeline is CRC32 and nothing else	6
6.3	The firmware archive	6
6.4	One private key inside a public distribution	6
6.5	Tail and signature: the verification key is not in the file	7
7	The live device: the shipped key is not used, and that changes the conclusion	7
8	Classification of the findings, and what to do	8
9	What was not established	9
10	How all of this was checked	10

11 The errors of the instruments in this work	11
12 Reproduction	12
A The numbers of this paper and their artifacts	12

1 Subject and scope: what was checked, and what it cannot be

There is exactly one subject: a firmware update file `BODYDATA.DAT`, 743 818 632 bytes (709.4 MiB), sha256 `dbdd8b02...c9eb`, md5 `ae4d074d...baac0`, sha1 `03c83a6b...36b2f`. No experiment modified the original: copies were made in `/tmp`, synthetic images were built separately.

The terms of reference (`TZ_SONY_ADUDIT.md`) ask, among other things, for the signature-verification mechanism on the device, the parser’s robustness against malformed input (fuzzing under emulation), rollback protection, behaviour when an update is interrupted, and whether debug interfaces are present in a release build. From a single file these items are **not checkable**: signature verification is done by code in the camera, and error handling by its bootloader. Everything below is marked as measured on the file; everything that concerns device behaviour is marked as not measured.

Two limits were observed without exception, and they are stated here because one of them is not what the word “static” alone would suggest. First, the analysis of the file is static, and it is complemented by exactly one bounded observation of the live device: TCP connects to three ports and a single SSH handshake, performed on the owner’s own camera, with no authentication, no payload written to the camera, and nothing sent to any other host (§7 reports it in full, including the fact that this observation yielded a negative result). Second, neither the paper nor the instruments shipped with it contain **any key material** — neither the body key’s value nor the private key. The instruments read the body key from the audit source at run time and print only *fingerprints*; the paper names fingerprints and the location where the key was published, never the key.

2 Method: the rules this work followed

Four rules shaped the work, and they explain the shape of the paper.

1. **Every claim gets a route independent of the one that produced it.** The container format was parsed with our own code and compared against an independent public implementation (`fwtool.py`); the body was unpacked by two implementations (our byte-wise one and the library’s) and the bytes compared; the key fingerprint was computed by our own P-256 arithmetic and checked with an outside `ssh-keygen`; the container CRC32 was recomputed by our own code.
2. **Every check has a control that must go red.** An instrument that cannot say “no” does not prove “yes”: a control is the same kind of check, phrased so that the correct answer to it is *failure*.
3. **Every number is bound to an artifact.** It is re-derived from the artifact’s bytes on every check of the paper (§10) instead of being quoted from a report.

4. **The errors of the instruments stay in the record.** This work produced 27 of them (F140–F166), none deleted; the ones that changed a *conclusion* rather than the wording are in §11.

3 The container: parsed to the byte, but not authenticated

The file is a sequence of blocks of the form “u32 length, 4-byte tag, payload” (the first field is the length of the *payload*, so a block’s body starts eight bytes after the block begins; Table 1). The parse agrees with an independent implementation: the tags DATV, PROV, UDID, FDAT and DEND sit where they should, and these blocks account for the whole file with nothing left over.

Table 1: Container blocks (the offset is the block’s start, i.e. its length field).

tag	offset	payload length	what it is
DATV	8	4	container version
PROV	20	4	provenance
UDID	32	60	USB descriptor table
FDAT	100	743 818 512	encrypted body (its body starts at 108)
DEND	743 818 620	4	CRC32 of everything before it

The container’s outer integrity is a CRC32: the stored value 0x1fafdcaa equals the crc32 of the first 743 818 620 bytes. That catches accidental corruption, but it is not authentication: a modified file with a recomputed CRC32 is accepted by the reference parser, while the same file without the recomputation is rejected (controls C8/C8c in the instrument checks.py).

The UDID table holds 7 entries of 8 bytes each (PID, VID, flags): six ordinary descriptors 054c:0448, 054c:047d, 054c:0d15, 054c:0d16, 054c:0d19, 054c:0d1a with flag 1, and a seventh with flag 2 whose PID is 03e2 (054c:03e2). What matters here is that the update-mode descriptor sits *inside the file itself* rather than being imported from an outside table: previously it was known only from the independent Sony-PMCA-RE project, and now it is confirmed from within.

4 The header is readable with a public key, and ECB leaks structure

The first 512 bytes of the body (that is, starting at offset 108) decrypt under the public key key_aes — the one that has lain in the open in a public implementation of the format for years — in **ECB** mode. Inside is a coherent structure: a 4-byte frame (c4a9fc03), the magic UDTRFIRM, format version 0100, mode U, flag N, model **0x91030083**, firmware version **5.01**, region 0, and the component layout (Table 2).

Two things in this section deserve their own paragraph.

(a) **The header’s integrity is not key-based either.** A crc32 over four trailing bytes is a bijection, so the stored checksum *determines* the bytes $H[508:512]$, and they are 00000000. A synthetic header that declares firmware.offset beyond the end of the file is silently

Table 2: The FDAT header: measured fields.

field	value
header checksum (crc32 over [12, 508))	0x9eb4163d
bytes $H[508:512]$ that this checksum requires	00 00 00 00
file system U	offset 512, size 1 253 376
slot P	offset 512, size 0 (no data)
firmware area	offset 1 253 888, size 739 658 240
number of file systems	2

accepted by the reference parser (control: with a broken CRC, it is rejected). The format and the reference implementation therefore carry no invariant that “offsets lie inside the image” (CWE-20/CWE-1284) — measured on the implementation, not on the camera’s parser, which we never saw.

(b) **ECB leaks structure, and this is a prediction rather than hindsight.** From the decrypted header one can see where a run of identical 16-byte ciphertext blocks *must* appear: **26** of them. Measuring the file itself, *without any key*, finds exactly **1** such run, starting at block **6** — that is byte **96** of the ciphertext — with length **26** blocks (**416** bytes) out of the 743 818 240 bytes of ciphertext. This is the ECB leak (CWE-327): an observer who does not know the key still learns the fact “416 bytes here are identical text”.

5 The body: opened with a public key; the “unaccounted bytes” were arithmetic

The body key — two components, `key_aes` plus the 32-byte generation key `key_cxd90057_k8` for CXD90057 — sits in a public but *unmerged* pull request against a public implementation of the format (author DavidBuchanan314, opened 10 September 2026). Confidentiality of the body is therefore broken publicly, and that is not our doing: we merely found a publication that was not in the main branch. Unpacking yields a stream of **740 912 128** bytes, sha256 11880291...1e66, reproduced independently by two different unpackers (our byte-wise one and the library’s) with identical bytes.

The stream is a sequence of 1024-byte frames: a 2-byte checksum, 2 bytes of size and end-of-stream flag, then the payload. Measured: **726 385** frames, all checksums valid (**0** bad), payload size **1020** bytes in every frame but the last (**448**), the end flag set for **1** frame and only the last, and a tail padding of **572** bytes, all `0xff`.

This closes a question that the first report carried as “2 906 112 unaccounted bytes” (hypothesis F-07): they are the frame overhead plus the padding, $4 \times 726\,385 + 572 = 2\,906\,112$. The mistake was not in the file but in my coordinates: the header’s offsets are in *stream* coordinates (after the frames are stripped), whereas the “remainder” was computed as the difference between the ciphertext length and the sum of the components. The stream equals $512 + 1\,253\,376 + 739\,658\,240 = 740\,912\,128$ bytes, leaving **0** extra bytes.

6 What is inside

6.1 The updater's file-system image

The first 1 253 376 bytes of the stream are a **squashfs** 4.0 image (ZLIB, built on 29 March 2024), sha256 62e86fad...d8ccaa. It was unpacked in full: **58** files, **3 151 747** bytes, of which **31** are ELF (all AArch64) and **21** are scripts. Roles were read from the files themselves rather than assigned by name: `edix.elf` partitions the storage, `pformat.elf` (**1 631 008** bytes) writes images and carries a full OpenSSL, including `AES_encrypt`, `SM4_encrypt`, `Camellia_cbc_encrypt`, `d2i_PKCS8_PRIV_KEY_INFO` and `CMS_EncryptedData_it`, with its own [Encrypt] Error: branch — image encryption on the device exists, and lives there. `config/chassis` reads CXD90057+CXD90058 (independent confirmation of the hardware class) and `config/body_version` reads 700.104.039.

6.2 The update pipeline is CRC32 and nothing else

`config/config.xml` in this image is the update map: every step is tied to its own checksum and the order is precisely *check the CRC32 of a group, then run the module*. There is no signature step in the pipeline. The internal `.sum` files are CRC32 too (we recomputed them over the members' own bytes; one changed byte yields a consistent new checksum without any key). Next to it sits `MODE_SERVICE=2`: a service mode exists, and in it `us_mkfs.sh` skips repartitioning and writes only the application image. And separately: in `loader_writer.sh` the md5 check of the loader is **commented out**, while `nor_loader_writer.sh` declares `loader_nor.md5` and never calls the check.

6.3 The firmware archive

The firmware area (739 658 240 bytes) is a tar of **159** members: partition configurations (`mount.conf`, built 29 June 2023 — that is the date of the *image build at Sony*, not of our work), partition images (WBI, `/usr = /dev/nflasha15`), regional files, i2c firmware, CA certificates inside the `/usr` image, and libraries' string tables. A note on method: **172** occurrences of the armour line `CERTIFICATE` in the stream are a count of *literals*, not of objects, and §6.4 shows where that can lead: of the **8** blocks that begin with `BEGIN PUBLIC KEY`, one turned out to be a string with no body. The file names its own model: `ILCE-7SM3` occurs **61** times (including the pair `ILCE-7SM3 v5.01`), and the only other model name, `ILCE-9`, occurs **2** times in a neighbouring table. The public tables that map a model code to a name do not contain this model: our code `0x91030083` is a different CXD90057-class device from the a7 IV (`0x01030081`).

6.4 One private key inside a public distribution

In the whole stream there are **5** lines beginning `BEGIN OPENSSH PRIVATE KEY` and the same number of `END` lines, **10** literals in total. Exactly one of them is a real key: the other four pairs are string tables in the `libcrypto/libssh2` libraries, where `BEGIN` and `END` stand tens of bytes apart. The distinguishing measurement is the body length: a real key has **435** base64 characters between the lines.

The key sits at offset **587 628 092** inside the member `0700_part_image/dev/nflasha5` (the /wbi partition image), in a block of **504** bytes. Parsing it gives `openssh-key-v1, cipher=none, kdf=none` (that is, it is *not* password-protected), `type ecdsa-sha2-nistp256, curve nistp256, comment root@(none)`. That it works was checked along three independent paths: our own P-256 arithmetic reproduces the public point from the private scalar; an outside `ssh-keygen` prints the same fingerprint `SHA256:J8L9aBLE...NODI`; and the sha256 of the public blob is `27c2fd68...c4d3832`. Control: flipping one bit of the scalar breaks $dG = Q$.

What the key is for can be read from the same file. The `sshd_config` member (`0800_appli/tmp/ssh/sshd_config`, **3094** bytes, at offset **740 733 952**) contains a *single* `HostKey` line, `HostKey /tmp_network/ssh/ssh_host_ecdsa_key`, and exactly four algorithm lines: `HostKeyAlgorithms ecdsa-sha2-nistp256`, `KexAlgorithms ecdh-sha2-nistp256`, `MACs hmac-sha2-256`, `Ciphers aes128-ctr`. Next to it we found the full text of the script that creates such a key — `/usr/bin/create_host_key.sh: ssh-keygen -q -b 256 -t ecdsa -N '' -f /tmp_network/ssh/ssh_host_ecdsa_key`. The comment `root@(none)` is exactly what `ssh-keygen` writes when run as root on a machine with no hostname.

The key grants no access: key-based login is **forbidden** in that config (`PubkeyAuthentication no`, `PermitRootLogin no`, `PasswordAuthentication no`, `MaxSessions 0`), and only forwards to `localhost:15740` and `localhost:60152` are allowed — it is a server identity for a tunnel, not a way in. Its risk runs the other way: the private half of a host key sits in a file every camera owner downloads, and those are the same bytes in every copy of firmware 5.01.

6.5 Tail and signature: the verification key is not in the file

After the ciphertext area (the tail is read at offset **743 818 348** inside `FDAT`) there remain **272** bytes: 16 bytes of plaintext IV (`89a85276c7208ea533a0e187fac4092c`) and **256** bytes — exactly the size of an RSA-2048 signature. The file does contain RSA public keys (6 distinct moduli: two of 3071 bits, three of 3072, one of 2048), and the 2048-bit one (at offset **429 029 376**) is the only one that could, by size, sign those 256 bytes. It does not: raising the tail to the power e modulo n produces no PKCS#1 v1.5 structure (the first two bytes are `49 7f`), and no PSS either. An exhaustive sweep (2 keys $\times$ 10 candidate messages $\times$ 4 hashes $\times$ {v1.5, PSS}) yields not one acceptance. The published Sony module from the Sony-PMCA-RE project fares no better.

The right conclusion is not “there is no signature” but “**there is nothing in the file with which to confirm one**”: no verification key is present, so it must live in the bootloader or in ROM. That is the trust boundary, and it lies outside the subject of this audit.

7 The live device: the shipped key is not used, and that changes the conclusion

The decisive check for §6.4 is a single one: read the host key of a real camera and compare. It was done — from two machines independently (the owner’s Mac and this container), against the device `192.168.1.102`, whose port 22 answered in **22** probes out of **22** over three minutes, while port 15740 (PTP/IP) was closed.

The result is **VERDICT: MISMATCH**. The live key is `SHA256:0V0xJqn75lYs4Lf91tSlZu8nazWWxD6rTVc35jZ087U` (blob sha256 `d153b126...3668f3b5`), against the firmware key

SHA256:J8L9aBLE...NODI. They differ in 64 of the 66 bytes of the point: a different key, not a typo. Three comparison routes were used (the blob's base64, the ssh-keygen fingerprint, the sha256 of the decoded blob), and all three said “no”.

But the camera said more than “my key is different”. In the same SSH exchange it negotiated *exactly* four algorithms, and that is precisely the set imposed by the `sshd_config` inside the firmware (§6.4). An ordinary machine does not choose such a set: in the owner's run, his own workstation on the same network (192.168.1.103, OpenSSH 10.3) negotiated `mlkem768x25519-sha256` and `chacha20-poly1305`, while two other devices with port 22 open turned out to be dropbear and negotiated no matching `kex` at all. Hence a conclusion worth more than the verdict itself: **the sshd running on the camera is the firmware's own, but its host key is its own too**. So the path `create_host_key.sh` really does execute on the device, and the private key inside the public file is *not* this camera's live identity.

What that changes and what it does not, stated plainly. It does not change: the key in the file remains a private key in an open distribution, and the leak does not disappear because the device does not use it. It does change: the question moves from “whose key is this” to “is the camera's identity verifiable at all” — and if the key is created afresh under `/tmp_network` on every boot (which is what the recovered script and the path in `sshd_config` say, and which we did *not* measure), then the answer is no. A client that honestly records the fingerprint in `known_hosts` will see a key-change warning every time the camera is switched on: the cryptography is present and the authentication is not, and the worst effect here is not cryptographic but human — it trains the operator to dismiss the warning.

What we did *not* measure, and do not claim: whether the live key changes across a reboot (it lives outside persistent storage, but that is a premise read from a config, not a measurement — it needs one run before and one after a power cycle), and whether it is the same on every camera of the model. And separately, as a fact without an explanation: the PTP/IP port 15740, which a camera normally opens, was closed in that state — so “the camera was found” rests here on the algorithm set and on the key match, not on the PTP/IP port.

8 Classification of the findings, and what to do

The recommendations follow from the findings rather than from general principle.

1. **Retire the shared header key.** One key across several camera generations means a single publication opens every header. Replace ECB with an authenticated mode (AES-GCM/SIV) keyed per model, so that integrity cannot be recomputed without the key.
2. **Base version decisions only on signed data.** A header that anyone can manufacture is no basis for deciding whether to install or to roll back: either include the version inside the signed area, or keep it out of the header entirely.
3. **Do not use CRC32 as a trust boundary**, and separately, **enforce bounds**: today the format does not require offsets to lie inside the image.
4. **Do not ship a private host key.** Generate it on the device in persistent storage; and do not keep it under `/tmp` if a client is ever to pin the camera's identity.

Table 3: Findings: what was measured, its class, and what follows.

class	finding	access type
CWE-327	a shared public header key in ECB mode; the block run leaks (26 blocks, predicted ahead of measurement)	local delivery of the file
CWE-345	container and inner integrity are CRC32, recomputable without a key	local delivery of the file
CWE-20/1284	the format does not require offset/size to lie inside the image; the reference parser accepts an offset past the end of the file	local delivery of the file
CWE-321/798	an unencrypted OpenSSH private key (ECDSA P-256) inside a publicly distributed image; the same bytes in every copy of 5.01. Measured <i>not</i> to be the camera’s live identity (§7)	hygiene; no vector demonstrated: the camera does not use this key, so impersonation would need a client that pinned it from elsewhere
CWE-1188 (<i>hypothesis</i>)	if the host key is created afresh under /tmp on every boot, the camera’s identity cannot be pinned by a client. Whether it survives a reboot was <i>not</i> measured	hypothesis; the decisive measurement is one run before and one after a power cycle
—	no signature step in the update pipeline; no verification key in the file	needs the device

5. **Anti-rollback in practice** (a monotonic counter in a protected area) is not checkable from a file, but structurally it is absent from the header.
6. **The next step needs a device, not a file:** a bootloader dump or service mode. Only with them do the remaining items of the terms of reference stop being “not checkable from a single file”.

9 What was not established

The list is kept without softening, because it is exactly what separates a report from advertising.

1. Signature verification on the camera: what is signed, by what, and whether it is checked before the image is applied.
2. The root of trust (bootloader/ROM): it is not in the file.
3. Rollback protection in practice; behaviour when an update is interrupted.
4. The robustness of the *camera’s* parser against malformed input: what we measured is a property of the format and of the *reference* implementation, not of the camera.
5. What the 256-byte tail formally is (its shape matches an RSA-2048 signature; there is nothing with which to confirm it).
6. Whether the live host key changes on reboot, and whether it is identical across units of the model.

7. The authenticity of the file itself: nothing inside it proves it came from Sony.
8. The reason port 15740 was closed in the measured state.

10 How all of this was checked

(1) **The audit instruments, run again.** Every existing check was re-run today against the unchanged file:

Table 4: Re-run of the audit instruments (all against the file with sha256 dbdd8b02...c9eb).

instrument	checks	controls	outcome
audit/checks.py	15/15	3/3 red	all consistent
audit/report_check.py	2/2	3/3 red	all consistent
audit/round277_check.py	13/13	5/5 red	all consistent
audit/round278_check.py	17/17	5/5 red	all consistent
audit/round279_check.py	28/29	2/2 red	one SKIP: the camera is offline now
audit/parser_bounds.py	—	—	format bounds, as described
	47/47	5/5 red	all consistent

The round279_check.py row is a measurement, not a caveat: the live half of that suite (C22–C25: the MISMATCH verdict, the algorithm set, the key matching the owner’s key) did *not* execute today, because the camera does not answer from this container; the instrument prints SKIP rather than a green. The live measurements were taken earlier and travel with the report and with this paper as separate witness files.

(2) **All findings re-verified by a separate instrument.** is not a retelling of the reports: it takes 47 statements and re-derives each one from raw bytes with its own code (its own container parse, its own frame unpacking, its own tar walk, its own openssh-key-v1 parse, its own P-256 arithmetic, its own crc32). Outcome: **47** non-control checks green out of **47**, and **5** controls red as they must be. The controls are: searching for a string that is definitely absent; a corrupted header prefix; a corrupted key scalar; a corrupted frame; a read past the end of the stream.

(3) **What the re-verification itself found: 4 corrections to our own reports.** This is the most important part of the section, because they concern numbers already shipped in delivered packages.

- **The key block is 504 bytes, not 501**, and its body is **435** base64 characters, not 471 (471 is the offset at which the closing line begins, read as a length).
- **The offset of sshd_config is 740 733 952, not 740 736 525** (the recorded value lands 2573 bytes *inside* the file; its size, 3094, was right).
- **There are 8 blocks beginning BEGIN PUBLIC KEY, not 4**, and **6** distinct RSA moduli, not 5 (two of 3071 bits, three of 3072, one of 2048). One of the eight blocks is a literal with no body — a string table, not a key; the same is true of the single literal BEGIN ENCRYPTED PRIVATE KEY.
- **The body of FDAT starts at byte 108** (the length field is at 100). Checking such an offset matters: a check that reads the header at byte 100 produces exactly the same *appearance* of a result (decryption “works”, hashes come out) on different bytes.

None of the corrections changed a conclusion; all four changed numbers that had been recorded. That is the price of the rule “every number is bound to an artifact”: while a number lives in prose, nobody re-measures it.

(4) **The numbers of this paper.** `paper_check.py` re-derives every number in the paper from the artifact’s bytes (the path and sha256 of each artifact are recorded in `numbers.json`), requires the number to appear both in this source and in the printed PDF, and `selftest_paper.py` proves the instrument can go red: on a corrupted number in the text, on a substituted value in a copy of an artifact, and on a deleted artifact. Appendix A is printed from `numbers.json` rather than typed, so the list of numbers cannot drift away from the check. A separate layout instrument (`layout_check.py`) checks the printed pages for letters lying over other letters and for ink outside the type area — a defect TeX reports only as an “Overfull hbox”, and not for every case. A third instrument, `string_check.py`, requires the 46 names these claims rest on to be present both in the source and in the text extracted from the printed PDF — it exists because names, unlike numbers, had gone missing from the paper without any check noticing (§11).

11 The errors of the instruments in this work

The work produced 27 of them; here are those that changed a conclusion, or that belong to a class worth recognising. The full list is in `FAILURES.md` and in the per-round reports.

- **An instrument counted instructions as artefacts.** The claim “the image has three OPENSSH PRIVATE KEY blocks” was a count of *literals* inside libraries’ string tables. The real key is one. Same class: “six public crypters” — the seventh lay in an unmerged pull request, invisible to a search of the main branch.
- **An instrument measuring a state it was itself changing.** The first version of the CRC-tampering check computed the checksum while the write had not yet been flushed to disk, and returned a false “the CRC cannot be forged”.
- **Precision of comparison.** The report checker compared numbers with a relative tolerance of 10^{-6} and therefore missed `739658240` → `739658241`. A comparison must be as precise as the text claims to be.
- **One false conclusion, reached twice.** Parsing the line `kex: host key algorithm:(no match)` yielded the word `match`), and `ssh-keyscan` terminates its lines with a carriage return; as a result the check twice declared “a different algorithm set” about a camera that had negotiated exactly the firmware’s set. It was caught only by running against the live device instead of against my model of a server.
- **An instrument that reddened by itself.** The control for the expected frequency of magic bytes compared three-byte patterns against `/dev/urandom` and failed in one run out of three for reasons unrelated to the subject: a control must be deterministic.
- **Lazy decryption.** The “wrong key” control was blind, because the library’s `decrypt()` is lazy and reads no bytes: it “accepted” a key of all zeros.
- **A check that looked at the wrong representation.** While building the number instrument of this paper, the payload offset was verified by comparing the file’s raw bytes against the plaintext magic — but that region is encrypted, so the check could only ever fail. The offset is now verified on the *decrypted* window.

12 Reproduction

1. Verify the subject: `sha256sum BODYDATA.DAT` must give `dbdd8b02...c9eb`. No audit command writes to that file.
2. Audit instruments: `audit/checks.py`, `audit/report_check.py`, `audit/round277_check.py`, `audit/round278_check.py`, `audit/verify_findings.py`; the live ones are `audit/round279_check.py` (needs `paramiko`) and `MAC_FINGERPRINT.sh` on a Mac.
3. Paper: `publication/run_all.sh` — it runs the instruments, rebuilds the numbers, builds the PDF, binds every number to its artifact, checks the layout, runs the self-tests and writes the manifest.

A The numbers of this paper and their artifacts

The table is printed from `numbers.json` by the generator `make_numbers.py`: it is the list of numbers that `paper_check.py` checks. The “artifact” column is the path whose bytes the number is re-derived from; the sha256 of every artifact is recorded in `numbers.json`.

Table 5: The numbers of the paper, grouped by the artifact whose bytes each one is re-derived from. The sha256 prefix of every artifact is printed once, at the group it belongs to; the full values are in `numbers.json`.

number	what it is
BODYDATA.DAT sha256 dbdd8b02ed81d6f0	
743 818 632 dbdd8b02ed81d6f0e0a0d36b55f5d12fa1bc f8682fbad1393996f10d1494c9eb ae4d074dca268917cc9697714f6baac0 03c83a6b616c9f216495437359fac585db336b2f	size of the update file, bytes sha256 of the object md5 of the object sha1 of the object
- sha256 -	
709.4	the same size in MiB
BODYDATA.DAT sha256 dbdd8b02ed81d6f0	
8	DATV block: offset
20	PROV block
32	UDID block
100	FDAT block
743 818 512	FDAT payload, bytes
743 818 620	DEND block: offset (= length of the prefix the CRC covers)
0x1fafdcaa	stored container CRC32
7	entries in the UDID table
0x03e2	last descriptor: PID of the updater mode
108	offset of the FDAT payload (= start of the body)
c4a9fc03	frame of the first body block
UDTRFIRM	body magic
0100	format version
0x9eb4163d	header CRC32
5.01	firmware version in the header
0x91030083	model code in the header
0	region
00000000	the bytes H[508:512] that the CRC32 requires
512	offset of file system U in the stream
1253376	size of file system U
0	size of slot P (empty)
1253888	offset of the firmware area
739658240	size of the firmware area
2	number of file systems in the header
1	runs of identical 16-byte ciphertext blocks (measured without the key)

number (continued)	what it is
6	start of the run, 16-byte block index
96	start of the run, bytes from the start of the ciphertext
26	length of the run, 16-byte blocks
416	length of the run, bytes
743818240	length of the ciphertext, bytes
out/stream.bin sha256 11880291ae4bd08b	
740912128	unpacked stream, bytes
11880291ae4bd08b61c9d08475ee8a33703b	sha256 of the unpacked stream
1c8f08ad35cbcd3a47174101e66	
- sha256 -	
726385	number of stream frames
0	frames with a bad checksum
1020	payload size of a frame
448	payload size of the last frame
1	frames carrying the end flag
572	padding of the last frame, bytes
2906112	frame overhead plus padding (= the former "unexplained" quantity)
0	bytes beyond the declared components
out/fs_user.sqsh sha256 62e86fad175e0326	
62e86fad175e0326969d5f1cea29d780feb4	sha256 of the squashfs image
96ce8034acd2c9c3954320d8ccaa	
out/fs_root sha256 tree:5cb85a8b2b2	
58	files in the unpacked updater image
3151747	total size of the unpacked files, bytes
31	of them ELF
21	of them scripts
out/fs_root/bin/pformat.elf sha256 3ce2a76f477e9373	
1631008	size of pformat.elf, bytes
- sha256 -	
159	members in the firmware tar
out/stream.bin sha256 11880291ae4bd08b	
61	occurrences of the string ILCE-7SM3 in the stream
2	occurrences of the string ILCE-9 in the stream
172	occurrences of the CERTIFICATE armor literal (literal count)
587628092	offset of the private-key block in the stream
504	size of the key block (BEGIN..END)
435	base64 characters in the key body
ecdsa-sha2-nistp256	key type
none/none	cipher and KDF (none)
root@(none)	comment inside the key
SHA256:J8L9aBLEimTdDW5m1iV7wU64LP1iW	key fingerprint (ssh-keygen)
o3+lJ9MGCxN0DI	
27c2fd6812c48a64dd0d6e66d6257bc14eb8	sha256 of the key public blob
2cfd625a8dfe963f4c182c4d3832	
5	BEGIN OPENSSH PRIVATE KEY lines (one of them a real key)
10	OPENSSH PRIVATE KEY literals (BEGIN+END)
8	BEGIN PUBLIC KEY blocks (one is a literal with no body)
8	SPKI blocks that parse as keys
6	distinct RSA moduli
429029376	offset of the 2048-bit key
740733952	offset of the sshd_config member in the tar table of contents
3094	size of sshd_config, bytes
BODYDATA.DAT sha256 dbdd8b02ed81d6f0	
272	body tail: total bytes
89a85276c7208ea533a0e187fac4092c	IV in the tail (in clear text)
256	signature block, bytes (= the size of an RSA-2048 signature)
743818348	offset of the tail in the file
497f	first bytes after raising the tail to the power e (no v1.5 structure)
publication/live_witness/FINGERPRINT_RESULT_2026-10-04_1743.txt sha256 411ec1397eb43f4f	

number (continued)	what it is
192.168.1.102 MISMATCH SHA256:0V0xJqn75lys4Lf91tSlZu8nazWWx D6rTVc35jZo87U d153b126a9fbe65cace0b7fdd6d4a566ef27 6b3596c43eab4d5737e63668f3b5	camera address in the runs verdict of the host-key comparison fingerprint of the camera live key sha256 of the live key blob
publication/live_witness/live_probe_log.txt	sha256 4cbdc8c67c01054c
22	probes that found port 22 open
- sha256 -	
22	camera sshd port
publication/live_witness/FINGERPRINT_RESULT_2026-10-04_1743.txt	sha256 411ec1397eb43f4f
SHA256:J8L9aBLEimTdDW5m1iV7wU64LP1iW o3+lJ9MGCxNODI	fingerprint of the firmware key it was compared against
- sha256 -	
64 66 15740	of the 66 bytes of the key point differ (external measurement) length of a P-256 point, bytes PTP/IP port (closed in the measured state)
publication/verification_runs/checks.txt	sha256 41efd26400c45698
15 3	checks.py: green checks.py: controls
publication/verification_runs/round277.txt	sha256 385c3674d0bfff57f
13 5	round277_check.py: green round277_check.py: controls red
publication/verification_runs/round278.txt	sha256 78d1932d25d8911b
17 5	round278_check.py: green round278_check.py: controls red
publication/verification_runs/round279.txt	sha256 0a665fe918405af3
28 29	round279_check.py: green round279_check.py: total
publication/verification_runs/verify_findings.txt	sha256 30af8e4f5c111ad5
47 5	verify_findings.py: green verify_findings.py: controls red
- sha256 -	
4 27	number of corrections to numbers recorded earlier instrument errors across the work (F140-F166)